

Scrum: Backlog produktu

Rozproszona współpraca agentów LLM

1. O projekcie i produkcji

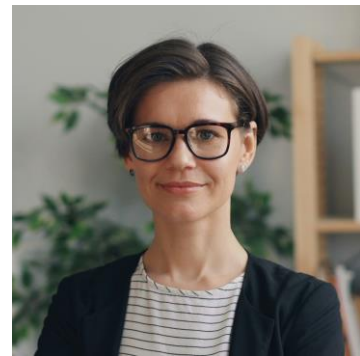
Projekt dyplomowy inżynierski “Rozproszona współpraca agentów LLM” jest kontynuacją projektu grupowego “Rozproszona współpraca LLM inspirowanna umysłami rojowymi”. Celem projektu jest implementacja systemu rozproszonej współpracy agentów LLM z koordynacją zdecentralizowaną oraz porównanie właściwości systemowych i jakości generowanych odpowiedzi podczas równoległej i rozproszonej współpracy agentów LLM.

2. Persony użytkowników

2.1. Naukowiec przeprowadzający badania dot. sztucznej inteligencji

2.1.1. Najważniejsze informacje

- **Imię i Nazwisko:** dr inż. Rita Mochocka
- **Wiek:** 36
- **Wykształcenie:** doktorat w dziedzinie informatyki teoretycznej, specjalizacja: modele językowe (LLM)
- **Zawód:** badaczka na uczelni technicznej
- **Stan cywilny / rodzinny:** niezamężna



2.1.2. Historia i charakterystyka

Rita pracuje na pełny etat na stanowisku badacza od 5 lat. W czasie wolnym Rita słucha podcastów kryminalnych i jeździ na rowerze. W pracy zajmuje się analizą i badaniem możliwości logicznego rozumowania modeli językowych. W tym celu przeprowadza eksperymenty, przygotowuje publikacje naukowe i aktywnie śledzi bieżące trendy i innowacyjne rozwiązania.

Podczas ostatniej serii badań Rita zauważyła, że systemy wykorzystujące pojedyncze modele często popełniają błędy w logicznym rozumowaniu podczas rozwiązywania zadań matematycznych. Skłoniło to ją do eksploracji rozwiązań korzystających z wielu współpracujących agentów. Jednym z wyzwań w tym przedsięwzięciu jest brak gotowych rozwiązań pozwalających na łatwe i powtarzalne porównanie możliwości różnych architektur modeli wieloagentowych. Szuka ona rozwiązania pozwalającego jej zebrać wiarygodne wyniki eksperymentów.

2.1.3. Zaawansowanie w wykorzystaniu technologii IT

Do przeprowadzania swoich badań i eksperymentów Rita korzysta z wydajnej stacji roboczej z systemem Windows 11, znajdującej się w jej gabinecie. Pracuje w Pythonie i używa narzędzi takich jak Jupyter Notebook czy Pycharm. Wykorzystuje lokalne modele językowe lub zasoby obliczeniowe swojej uczelni podczas przeprowadzania bardziej wymagających eksperymentów. Rita często tworzy własne skrypty do automatyzacji przeprowadzanych testów. Do organiznacji swojej pracy i analizy otrzymywanych wyników wykorzystuje takie narzędzia jak Git, GitHub, LaTeX czy Excel.

2.1.4. Cele

- Przeprowadzanie eksperymentów mających na celu analizę porównawczą jakości odpowiedzi generowanych przez różne konfiguracje współpracujących modeli w porównaniu do pojedynczego modelu
- Porównanie jakości generowanych odpowiedzi dla współpracy równoległej i rozproszonej modeli

2.1.5. Potrzeby

- Znalezienie zestawu testowego problemów matematycznych służących jako benchmark do analizy jakości wygenerowanych odpowiedzi
- System powinien posiadać możliwość automatycznej oceny poprawności i jakości wygenerowanej odpowiedzi
- System powinien dysponować możliwością ustawienia testowanej konfiguracji współpracy modeli
- System powinien udostępniać metryki jakości (np. czas, stabilność, trafność)

2.1.6. Problemy i obawy

- Obawa, że interfejs systemu będzie nieintuicyjny i uciążliwy do nawigacji
- Obawa, że korzystanie z systemu będzie wymagać poświęcenia dużej ilości czasu do zrozumienia jego działania
- Obawa, że istnieć będzie problem z powtarzalnością eksperymentów
- Obawa, że system nie będzie kompatybilny z modelem, na którym Rita będzie chciała przeprowadzać badania

2.2. Pracownik działu R&D firmy technologicznej

2.2.1. Najważniejsze informacje

- **Imię i Nazwisko:** Leon Sadura
- **Wiek:** 32
- **Wykształcenie:** magister inżynier informatyki, licencjat z zarządzania
- **Zawód:** Senior Machine Learning Engineer
- **Stan cywilny / rodzinny:** żonaty



2.2.2. Historia i charakterystyka

Leon jest doświadczonym inżynierem w dziedzinie AI. W czasie wolnym czyta literaturę fantastyczną i chodzi na spacer. W pracy zajmuje się projektowaniem systemów opartych na modelach językowych mających na celu rozwiązywanie złożonych problemów wymagających wnioskowania logicznego. Dzięki swojemu wykształceniu potrafi łączyć aspekty techniczne projektów z wymaganiami biznesowymi dotyczącymi efektywności i kosztów projektowanych przez niego rozwiązań.

Ostatnio Leon zauważył, że zaprojektowany przez niego system wykorzystujący pojedynczy model językowy często podaje przekonujące ale błędne odpowiedzi. Sadura chciałby poprawić swój system wykorzystując współpracujące ze sobą modele. Jego głównym problemem jest brak narzędzia pozwalającego na łatwe porównanie różnych konfiguracji tej współpracy. Szuka on rozwiązania pozwalającego wiarygodnie ocenić, jaki model współpracy między agentami LLM będzie najbardziej korzystny.

2.2.3. Zaawansowanie w wykorzystaniu technologii IT

Leon pracuje na wydajnej stacji roboczej lub laptopie z systemem Windows 11. Korzysta również z infrastruktury chmurowej swojej firmy i ze środowisk kontenerowych (Docker). Pracuje w Pythonie, używając narzędzi takich jak Pycharm. Wykorzystuje lokalne modele językowe oraz zasoby obliczeniowe swojej firmy. Do organizacji swojej pracy i analizy otrzymywanych wyników wykorzystuje takie narzędzia jak Git, GitHub, Word czy Excel.

2.2.4. Cele

- Sprawdzenie czy jakaś konfiguracja współpracujących ze sobą modeli LLM poprawi jakość otrzymywanych wyników
- Porównanie kosztu i jakości wybranych konfiguracji współpracujących ze sobą modeli

2.2.5. Potrzeby

- Znalezienie zestawu testowego zawierającego zadania o jednoznacznych wynikach
- System powinien dysponować możliwością ustawienia różnych konfiguracji współpracy modeli LLM
- System powinien posiadać możliwość automatycznej oceny poprawności otrzymanej odpowiedzi
- System powinien udostępniać metryki jakości (np. czas generowania odpowiedzi, stabilność, trafność)

2.2.6. Problemy i Obawy

- Modele LLM często popełniają błędy przy zadaniach wymagających logicznego rozumowania
- Obawa, że interfejs systemu będzie nieintuicyjny i będzie wymagał dłuższego czasu do nauki obsługi systemu
- Obawa, że system będzie niekompatybilny z modelami, które Leon będzie chciał wykorzystać podczas projektowania swoich rozwiązań
- Obawa, że testowanie różnych konfiguracji zajmie dużo czasu, a co za tym idzie nie będzie opłacalne pod względem biznesowym

3. Scenariusz użycia produktu

3.1. Scenariusz: Obserwacja działania i benchmark aplikacji

Rita chce przeprowadzić analizę systemu z kilkoma współpracującymi modelami LLM. Użyje w tym celu aplikacji *Rozproszona współpraca agentów LLM*. Po ściągnięciu na komputer kodu źródłowego i przygotowaniu aplikacji do działania wraz z włączeniem opcji pokazywania logów, Rita uruchamia aplikację.

Na początku by przetestować podaje zapytanie “*Find a rational approximation of π with denominator less than 1,000*”. Obserwuje działanie *Researchera* który znajduje podstawy teoretyczne rozwiązywanego zadania i podaje wzory które będą używane przez *Calculators*. Po pewnym czasie widzi jakie odpowiedzi obliczyły poszczególne *Calculators* i otrzymuje wynik podany przez *Evaluator*.

Po własnym przetestowaniu aplikacji Rita używa wbudowanej funkcji benchmark, która po kolei sprawdza wiele zapytań z różnych dziedzin matematyki i pokazuje jakość otrzymanych odpowiedzi dla rozwiązanych zadań.

4. Backlog produktu

4.1. Lista zestawiająca elementy backlogu produktu

ID	Title	Labels	Status
1 PB-01	[PB-01] Implementacja równoległej współpracy agentów (wielowątkowość) #8	1 Funkcjonalna	In Progress
2 PB-03	[PB-03] Opracowanie zestawu zaawansowanych pytań testowych #10	1 Analityczna	In Progress
3 PB-04	[PB-04] Optymalizacja logiki Agenta Ewaluatora #11	2 Funkcjonalna	Todo
4 PB-02	[PB-02] Dodanie większej ilości ról kalkulatorów #9	3 User Story	In Progress
5 PB-05	[PB-05] Dodanie możliwości wyboru roli agenta kalkulatora #12	3 Funkcjonalna	Todo
6 PB-06	[PB-06] Implementacja graficznego interfejsu użytkownika (GUI) #13	3 User Story	Todo

4.2. Wyjaśnienie jednostek i skali priorytetów

- **1** - Priorytet wysoki, zadanie które jest wymagane do ukończenia projektu i/lub poprawnego działania aplikacji
- **2** - Priorytet średni, w większości zadania które mają usprawnić działanie aplikacji, ale nie są wymagane do jej poprawnego działania
- **3** - Priorytet niski, zadania które nie są wymagane i mają na celu polepszenie komfortu pracy z aplikacją
- **Funkcjonalne** - Konkretna operacja która musi zostać wykonana przez system
- **Analityczna** - Funkcja związana z analizą wyników
- **User Story** - Korzyść dla konkretnej osoby

4.3. Wyjaśnienie posortowania listy elementów w backlogu

W pierwszej kolejności lista posortowana jest po priorytecie danego zadania. Następnie sortowana jest po statusie zadań.

Zadanie z najwyższym priorytetem będące w trakcie wykonywania będzie na pierwszej pozycji listy

5. Kryteria akceptacji

5.1. Kryteria akceptacji do wdrożenia całego produktu

Produkt spełnia minimalne wymagania jakościowe, przechodzi testy dotyczące poprawności generowanych wyników i zostaje zatwierdzony przez promotora projektu dyplomowego.

5.2. Kryteria akceptacji wybranych elementów backlogu produktu

5.2.1. [PB-01] Implementacja równoległej współpracy agentów (wielowątkowość)

[PB-01] Implementacja równoległej współpracy agentów (wielowątkowość) #8

Open

Kacper2711 opened 2 hours ago · edited by Alicja-Sob

Edits Collaborator

User Story
Jako Programista, chcę aby agenci kalkulatorzy pracowali równolegle, aby skrócić całkowity czas oczekiwania na odpowiedź systemu oraz umożliwić niezależne generowanie wyników przez modele.

Szczegóły

- Wykorzystanie biblioteki threading lub multiprocessing w języku Python.
- Zapewnienie, że agenci kalkulatorzy (Llama 3.1:8b) wysyłają zapytania do lokalnej instancji Ollama w tym samym czasie.
- Synchronizacja dostępu do współdzielonego stringa, w którym przechowywane są wyniki.

Kryteria Akceptacji

- ☐ System wywołuje jednocześnie przynajmniej dwóch agentów kalkulatorów. ...
- ☐ Logi konsoli potwierdzają, że obliczenia rozpoczęły się i zakończyły w nakładających się przedziałach czasowych. ...
- ☐ Każdy z agentów poprawnie dokłada swoją propozycję wyniku do współdzielonego zasobu bez nadpisywania danych innych agentów. ...
- ☐ System zwraca błąd lub ponawia próbę, jeśli jeden z wątków ulegnie awarii. ...

Create sub-issue

5.2.2. [PB-03] Opracowanie zestawu zaawansowanych pytań testowych

[PB-03] Opracowanie zestawu zaawansowanych pytań testowych

Open

Kacper2711 opened 2 hours ago · edited by Alicja-Sob

Edits Collaborator

User Story
Jako Programista, chcę przygotować bazę trudnych zadań matematycznych wraz z wzorcowymi rozwiązaniami, aby przeprowadzić rzetelną analizę porównawczą systemu w wersji rozproszonej i równoległej.

Szczegóły

- Zebrać minimum 20 złożonych problemów matematycznych (algebra, geometria, teoria liczb).
- Przygotowanie pliku benchmarks.json zawierającego prompt oraz poprawny wynik dla każdego zadania.
- Uwzględnienie zadań, w których pojedynczy model Llama 3.1:8b często wykazuje tendencję do halucynacji.

Kryteria Akceptacji

- ☐ Każde pytanie w zestawie posiada zdefiniowane kryterium poprawności (wynik wzorcowy). ...
- ☐ Zbiór zawiera zadania o różnym stopniu skomplikowania (od prostych ułamków po geometrię czworościanu). ...
- ☐ Skrypt testowy automatycznie porównuje wynik Agenta Ewaluatora z wynikiem wzorcowym. ...

Create sub-issue

5.2.3. [PB-04] Optymalizacja logiki Agentu Ewaluatora

[PB-04] Optymalizacja logiki Agentu Ewaluatora #11

Open

Kacper2711 opened 2 hours ago · edited by Alicja-Sob

Edits Collaborator

User Story

Jako Użytkownik, chcę aby system inteligentnie odrzucał błędne wyniki i wymuszał ponowne obliczenia tylko wtedy, gdy jest to konieczne, aby oszczędzać zasoby sprzętowe i czas.

Szczegóły

- Implementacja algorytmu identyfikacji wyników zbliżonych i odrzucania wartości odstających w Agencie Ewaluatorze.
- Ustalenie limitu wywołań (iteracji), aby uniknąć nieskończonych pętli w przypadku braku konsensusu.

Kryteria Akceptacji

- ☐ Ewaluator poprawnie rozpoznaje wynik jako poprawny, gdy przynajmniej dwóch agentów kalkulatorów poda tę samą wartość (np. 355/113).
- ☐ System zwraca status #not_good i ponawia krok obliczeniowy, gdy wyniki są zbyt rozbieżne.
- ☐ Po przekroczeniu zadanego limitu prób (np. 3), system kończy działanie z czytelnym komunikatem o błędzie.

Create sub-issue

5.2.4. [PB-06] Implementacja graficznego interfejsu użytkownika (GUI)

[PB-06] Implementacja graficznego interfejsu użytkownika (GUI)

Open

Kacper2711 opened 2 hours ago · edited by Alicja-Sob

Edits Collaborator

User Story

Jako Badacz AI, chcę korzystać z interfejsu graficznego, aby móc wygodnie wprowadzać zapytania, obserwować proces współpracy agentów w czasie rzeczywistym oraz przeglądać sformatowane wyniki bez konieczności analizowania surowych logów w konsoli.

Szczegóły

- Wybór frameworka: Wykorzystanie biblioteki Python dedykowanej dla aplikacji AI.
- Wizualizacja procesu: Zaprojektowanie widoku, który oddziela działania poszczególnych ról: Badacza, Kalkulatorów i Ewaluatora.
- Monitorowanie stanu: Dodanie paska postępu lub dynamicznych powiadomień informujących o aktualnym statusie (np. „Badacz szuka wzorów...”, „Kalkulatory liczą...”)
- Prezentacja wyników: Wyświetlanie finalnej odpowiedzi w czytelnej formie (np. z użyciem LaTeX dla wzorów matematycznych).

Kryteria Akceptacji

- ☒ Użytkownik może wprowadzić prompt w dedykowanym polu tekstowym i zatwierdzić go przyciskiem.
- ☒ Interfejs wyświetla logi każdego agenta w osobnych, czytelnych sekcjach.
- ☐ Finalny wynik (zaakceptowany przez Ewaluatora) jest wyraźnie wyróżniony na ekranie.
- ☐ Aplikacja nie „zamraża się” podczas oczekiwania na odpowiedź z lokalnej instancji Ollama.

Create sub-issue

6. Definicja ukończenia

Element backlogu produktu uzajemy za ukończony gdy:

- Napisano kod
- Napisany kod został przetestowany i poprawione zostały ewentualne błędy
- Napisany kod umieszczony został w repozytorium
- Kod został zaakceptowany przez promotora projektu dyplomowego